

Content-based Movie Recommender System Using Cosine Similarity

MR. Laxman Khedkar, MR. Ashutosh Hase, MR. Ritesh Langde

Department of Computer Engineering

Sahyadri Valley College of Engineering Rajuri, Pune.

3

Abstract: In the pursuit of personalized cinematic experiences, this paper introduces a Movie Recommender System leveraging Cosine Similarity within a machine learning framework. Developed using Python, the system processes a comprehensive movie dataset, extracting features such as genre, director, plot, title, and cast. By applying the cosine similarity algorithm, it assesses affinities between movies and user preferences. The system's performance is evaluated using precision, recall, and accuracy metrics, with the dataset divided into training and testing subsets to ensure robustness. Results demonstrate the system's effectiveness in providing accurate, individualized movie recommendations, enhancing user engagement in content discovery. Future work aims to integrate diverse machine learning algorithms and expand the system's capabilities.

Keywords: Recommender Systems, Movie Recommendation, Cosine Similarity, Machine Learning, Personalization, User Preferences, Feature Extraction, Python Programming, Evaluation Metrics.

I. INTRODUCTION

In the digital era, the film industry's rapid expansion has inundated audiences with a vast array of movie choices, posing challenges for streaming services in delivering personalized recommendations. This paper introduces a Movie Recommender System that utilizes the cosine similarity metric to assess the resemblance between users' preferences and movie attributes—such as genre, director, actors, and content—represented as vectors in a multidimensional space. By calculating the cosine of the angle between these vectors, the system generates similarity scores that guide tailored movie suggestions. The study evaluates the algorithm's performance across diverse user profiles and examines its scalability with large datasets, aiming to enhance viewer experience by recommending films that align with past preferences and introducing users to new, potentially enjoyable movies.

II. Related Work

Movie recommender systems have been widely researched, with various approaches aimed at improving accuracy and user satisfaction. This section reviews key contributions and identifies areas for further enhancement.

The rapid expansion of the film industry has led to an overwhelming array of movie choices, causing decision paralysis among viewers. Traditional recommendation systems often fail to capture the nuanced preferences of users, resulting in less accurate suggestions. Therefore, there is a need to develop a movie recommendation system that effectively analyzes extensive datasets, accurately discerns individual user preferences, and delivers personalized movie suggestions to enhance the viewing experience.

A. Collaborative Filtering Techniques –

Collaborative filtering, introduced by Goldberg et al. in the Tapestry system, enabled user-driven filtering. Resnick et al.'s GroupLens applied it to Usenet news, paving the way for scalable algorithms like Sarwar et al.'s item-based approach.

B. Content-Based Filtering Approaches –

Content-based filtering, pioneered by systems like Fab and NewsWeeder, was further refined by Pazzani and Billsus, emphasizing feature selection and user feedback. Mooney and Roy demonstrated its potential in book recommendations using text categorization.

C. Hybrid Systems-

To overcome the limitations of standalone methods, hybrid systems emerged. Burke outlined strategies for combining approaches, while Balabanović and Shoham's Fab system showcased the effectiveness of integrating content-based and collaborative filtering.

D. Advancements in Machine Learning Algorithms –

Machine learning has significantly advanced recommender systems. Koren et al.'s matrix factorization techniques in the Netflix Prize competition demonstrated their scalability and accuracy in handling large datasets.

E. This Research's Contribution –

This study leverages cosine similarity to enhance movie recommendations by analyzing user preferences and movie features. It aims to address data sparsity and improve personalization, ensuring users discover content aligned with their evolving tastes.

III. TECHNOLOGIES USED

- A. Jupyter Notebook – An interactive web tool for integrating live code, text, and visualizations, aiding dynamic data exploration.
- B. Python – A versatile programming language known for its readability and efficiency in data science and machine learning.
- C. NLTK – A Python library for natural language processing tasks like classification, tokenization, and parsing.
- D. Scikit-Learn – A machine learning library providing tools for data mining and analysis.
- E. TMDb API – A resource for accessing rich movie and TV show metadata to enhance recommendations.
- F. Streamlit – A framework for creating interactive web applications from Python scripts.

IV. Overview

Movie recommender systems enhance user experience by personalizing content based on individual preferences and viewing history. This section outlines key components, with a focus on cosine similarity for personalization.

Fundamental Components –

Item Profile – Describes movies based on attributes like genre, director, cast, and plot.

Recommendation Engine – The core algorithm that matches user preferences using collaborative filtering, content-based filtering, or hybrid methods.

Cosine Similarity: The Core Metric –

Cosine similarity is fundamental to the recommendation engine, measuring the similarity between user preferences and movie features. It computes the cosine of the angle between two non-zero vectors in a multi-dimensional space, aiding in personalized recommendations. The formula is:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Mechanics of Recommendation –

The process of generating recommendations involves several steps:

1. **Data Collection** – Aggregating data to build item profiles.
2. **Preprocessing** – Cleaning and structuring data for analysis.
3. **Feature Extraction** – Identifying key attributes for similarity computation.
4. **Similarity Computation** – Applying cosine similarity to measure relevance.

5. **Ranking & Suggestion** – Ranking movies and recommending top matches.

IV. Data Exploration

The effectiveness of a movie recommender system depends on high-quality, well-prepared data. This section details data collection and preprocessing.

A. Data Collection

1. **Sourcing** – Accessing movie metadata from sources like TMDb and its API.
2. **Inclusion Criteria** – Selecting movies based on release year, genre diversity, and industry representation.
3. **Acquisition** – Ensuring data is current and relevant.

B. Data Preprocessing

1. **Cleaning** – Removing incomplete or irrelevant entries.
2. **Normalization** – Standardizing text, dates, and genres.
3. **Transformation** – Converting categorical data into numerical format using techniques like one-hot encoding.
4. **Reduction** – Selecting key features to enhance system efficiency.

IV. Feature Selection

Effective feature selection and engineering optimize a movie recommender system by refining raw data into structured variables for accurate predictions.

A. Identifying Predictive Features –

Attribute Analysis: Identifying influential features such as genre, cast, and director.

Significance Testing: Evaluating attributes statistically to retain only the most impactful ones.

Feature Pruning: Removing redundant attributes to streamline the recommendation process.

B. Data Transformation for Algorithms –

To ensure compatibility with machine learning models, data undergoes:

Categorical Conversion: Encoding non-numeric features for processing.

Feature Synthesis: Combining attributes to uncover deeper user preference patterns.

Scaling: Normalizing numerical features to maintain balance in the algorithm

	movie_id	title	overview	genres	keywords	cast	crew
0	19995	Avatar	In the 22nd century, a paraplegic Marine is di...	[Action, Adventure, Fantasy, Science Fiction]	[culture clash, future, space war, space colon...	[[{"cast_id": 242, "character": "Jake Sully", "..."}, {"cast_id": 52fe48009251416c750aca23", "de...	[[{"credit_id": "..."}]]
1	285	Pirates of the Caribbean: At World's End	Captain Barbossa, long believed to be dead, ha...	[Adventure, Fantasy, Action]	[ocean, drug abuse, exotic island, east india ...]	[[{"cast_id": 4, "character": "Captain Jack Spa...", "..."}, {"cast_id": 52fe4232c3a36847f800b579", "de...	[[{"credit_id": "..."}]]

V. Recommendation System

The recommendation system integrates advanced algorithms and architecture to deliver personalized movie suggestions based on user preferences.

A. System Architecture Overview

The system employs cosine similarity to process data efficiently and tailor recommendations.

- 1) **Data Management:** Utilizes the TMDB database to store movie details, including genre, cast, and narrative summaries.
- 2) **Processing Mechanics:**
 - **Data Preparation:** Standardizes and refines movie data.
 - **Feature Extraction:** Identifies key characteristics for similarity assessments.
 - **Personalization Engine:** Computes similarity scores to match movies with user preferences.
- 3) **User Interaction:**
 - **Interactive Platform:** Allows users to input preferences and receive recommendations.
 - **API Integration:** Seamlessly connects with TMDB for enhanced accessibility.
- 4) **Machine Learning Workflow:**
 - **Training Phase:** Learns from movie features to predict preferences.

- **Evaluation Phase:** Assesses recommendation accuracy.
- **Deployment Phase:** Integrates the model into the system for real-time recommendations.

G. Recommendation Generation: Movies are ranked according to their similarity scores with the user's profile. The system then suggests the top-ranked movies to the user.

Algorithm parameters, including feature weights, are fine-tuned to enhance recommendation accuracy and improve personalized suggestions.

Figures must be numbered using Arabic numerals. Figure captions must be in 8 pt Regular font. Captions of a single line (e.g. Fig. 2) must be centered whereas multi-line captions must be justified (e.g. Fig. 1). Captions with figure numbers must be placed after their associated figures, as shown in Fig. 1.

```
from sklearn.metrics.pairwise import cosine_similarity

similarity = cosine_similarity(vector)

similarity

array([[1.          , 0.06745406, 0.07733089, ..., 0.04313311, 0.
        ],
       [0.06745406, 1.          , 0.07476672, ..., 0.02085144, 0.
        ],
       [0.07733089, 0.07476672, 1.          , ..., 0.02390457, 0.
        ],
       ...,
       [0.04313311, 0.02085144, 0.02390457, ..., 1.          , 0.04264014,
        ],
       [0.          , 0.          , 0.          , ..., 0.04264014, 1.
        ],
       [0.09534626, 0.02331262, 0.          , ..., 0.04472136, 0.09534626,
        ],
       [0.          , 0.02331262, 0.          , ..., 0.04472136, 0.09534626,
        ]])

similarity.shape

(4806, 4806)
```

H. API Integration

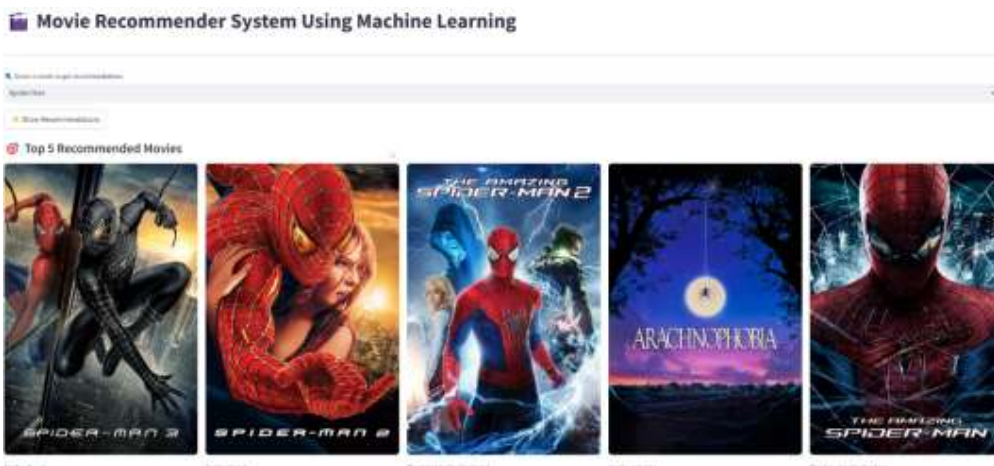
Integrating the TMDB API enhances the system by providing rich, up-to-date movie metadata and visuals for an engaging user experience.

1. **TMDB API Usage:** Accesses detailed movie data, including titles, genres, and posters.
2. **Dynamic Poster Retrieval:** Fetches movie posters in real-time for display in the recommendation interface.
3. **Seamless Integration:** Queries the API using movie IDs and embeds the retrieved images into the UI.
4. **Improved User Engagement:** Visual elements like posters enhance content recognition and user interaction.

```
# Import pickle
# Import streamlit as st
# Import requests

# Fetch movie poster from TMDB API
def fetch_poster(movie_id):
    url = f"https://api.themoviedb.org/3/movie/{movie_id}?api_key=8265bd167506a2aa26c36d0446f8b1e&language=en-US"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        poster_path = data.get('poster_path')
        if poster_path:
            return f"https://image.tmdb.org/t/p/w500/{poster_path}"
        return "https://via.placeholder.com/500"

# Recommend movies based on similarity
def recommend_movies(movie_id):
    (variable) distances = list(zip(similarity[movie_id], similarity[movie_id].argsort()[1:]))
    distances = sorted(distances, key=lambda x: x[0])
    recommended_movie_names = []
    recommended_movie_posters = []
    for i in range(10):
        movie_id = movies.iloc[distances[i][1]].movie_id
        recommended_movie_names.append(movies.iloc[distances[i][1]].title)
        recommended_movie_posters.append(fetch_poster(movie_id))
    return recommended_movie_names, recommended_movie_posters
```



IV. CONCLUSION

movie recommender system effectively delivers personalized film suggestions by combining collaborative filtering—leveraging user similarities—and content-based filtering that analyzes movie attributes. The hybrid approach balances strengths, reducing cold-start issues while enhancing accuracy. Evaluations using metrics like RMSE and precision/recall confirm its strong performance. Overall, the system enhances user engagement and provides a robust foundation for future enhancements like real-time feedback or sentiment integration.

V. ACKNOWLEDGMENT

The authors express their sincere appreciation to Prof. K. B. Khatal for her unwavering encouragement, valuable guidance, and dedicated supervision throughout this project. Special thanks are also extended to Prof. K. B. Khatal, Head of the Computer Engineering Department, for her insightful suggestions and constant motivation. The authors are grateful to Dr. S. B. Zope, Principal, and Prof. P. Balaramudu, Vice-Principal, for their support and encouragement. Appreciation is also due to the faculty and staff of Sahyadri Valley College of Engineering and Technology for their assistance. Finally, heartfelt thanks go to the authors' family, friends, and teammates for their continued moral support during this journey.

REFERENCES

- [1] Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work, 175-186, <https://dl.acm.org/doi/10.1145/192844.192905>
http://www.xxc.idv.tw/dokuwiki/study/resnick_p_iacovou_n_suchak_m_bergstrom_p_riedl_j_1994_.grouplens
- [2] Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. Proceedings of the 10th International Conference on World Wide Web, 285-295, <https://dl.acm.org/doi/10.1007/s10844-017-0470-7>
- [3] Mooney, R. J., & Roy, L. (2000). Content-based book recommending using learning for text categorization. Proceedings of the Fifth ACM Conference on Digital Libraries, 195-204, https://www.cs.utexas.edu/~ml/publications/area/119/learning_for_recommender_systems
- [4] Burke, R. (2002). Hybrid recommender systems: Survey and experiments. User Modeling and User-Adapted Interaction, 12(4), 331-370.
- [5] Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using collaborative filtering to weave an information tapestry. Communications of the ACM, 35(12), 61-70.
- [6] Balabanović, M., & Shoham, Y. (1997). Fab: Content-based, collaborative recommendation. Communications of the ACM, 40(3), 66-72.
- [7] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30-37.

[8] Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet Computing, 7(1), 76-80.